

Introduction



This dataset was scraped from nextspaceflight.com and includes all the space missions since the beginning of Space Race between the USA and the Soviet Union in 1957!

Install Package with Country Codes

```
%pip install iso3166
```

```
Requirement already satisfied: iso3166 in /usr/local/lib/python3.12/dist-packages (2.1.1)
```

Upgrade Plotly

Run the cell below if you are working with Google Colab.

```
%pip install --upgrade plotly
```

```
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (6.5.0)  
Requirement already satisfied: narwhals>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from plotly) (2.13.0)  
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from plotly) (25.0)
```

Import Statements

```
import numpy as np  
import pandas as pd  
import plotly.express as px  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
# These might be helpful:  
from iso3166 import countries  
from datetime import datetime, timedelta
```

Notebook Presentation

```
#set all float data to display only 2 decimal places  
pd.options.display.float_format = '{:,.2f}'.format
```

Load the Data

```
df_data = pd.read_csv('mission_launches.csv')
```

▼ Preliminary Data Exploration

- What is the shape of `df_data`?
- How many rows and columns does it have?
- What are the column names?
- Are there any NaN values or duplicates?

```
#check the shape of the data. There are 4324 rows and 9 columns
df_data.shape
```

```
(4324, 9)
```

```
#View the first 5 rows of data and the column names
df_data.head()
```

	Unnamed: 0.1	Unnamed: 0	Organisation	Location	Date	Detail	Rocket_Status	Price	Mission_Status
0	0	0	SpaceX	LC-39A, Kennedy Space Center, Florida, USA	Fri Aug 07, 2020 05:12 UTC	Falcon 9 Block 5 Starlink V1 L9 & BlackSky	StatusActive	50.0	Success
1	1	1	CASC	Site 9401 (SLS-2), Jiuquan Satellite Launch Ce...	Thu Aug 06, 2020 04:01 UTC	Long March 2D Gaofen-9 04 & Q-SAT	StatusActive	29.75	Success
					Tue Aug				

```
#view the last 5 rows of data
df_data.tail()
```

	Unnamed: 0.1	Unnamed: 0	Organisation	Location	Date	Detail	Rocket_Status	Price	Mission_Status
4319	4319	4319	US Navy	LC-18A, Cape Canaveral AFS, Florida, USA	Wed Feb 05, 1958 07:33 UTC	Vanguard Vanguard TV3BU	StatusRetired	NaN	Failure
4320	4320	4320	AMBA	LC-26A, Cape Canaveral AFS, Florida, USA	Sat Feb 01, 1958 03:48 UTC	Juno I Explorer 1	StatusRetired	NaN	Success
				LC-18A, Cape	Fri Dec 06				

```
#View 5 random rows of data
df_data.sample(50)
```


	Unnamed: 0.1	Unnamed: 0	Organisation	Location	Date	Detail	Rocket_Status	Price	Mission_Status
2804	2804	2804	RVSN USSR	Site 1/5, Baikonur Cosmodrome, Kazakhstan	Tue Jul 06, 1976 12:08 UTC	Soyuz Soyuz 21	StatusRetired	NaN	Success

✓ Data Cleaning - Check for Missing Values and Duplicates

Consider removing columns containing junk data.

				Site 90/20, Baikonur	Sat Mar 21, 1981	Tsvklon-2 I				
#check to see if the data contains any missing (nan) values										
df_data.isna().values.any()										
np.True_				ELS, Guiana Space	Fri Aug 22, 2014	Soyuz ST-B/Fregat-MT		StatusActive	NaN	Partial Failure
542	542	542	Arianespace	Centre, French	12:27	Galileo FOC				

```
#check to see how many missing values the data contains
df_data.isna().values.sum()
```

np-1nt64 (3366)	641	641	ULA	Canaveral AFS, Florida, USA	29, 2012 13:15 UTC	Delta IV Heavy NROL-15	StatusActive	350.0	Success
-----------------	-----	-----	-----	--------------------------------	--------------------------	-----------------------------	--------------	-------	---------

```
#check to see if the data contains any duplicate rows
df_data.duplicated().values.any()
```

np.False_	Russia	00.37 UTC
-----------	--------	-----------

```
#view details about the data. See that Price is the only column with missing values
df.data.info()
```

<pre><class 'pandas.core.frame.DataFrame'> RangeIndex: 4324 entries, 0 to 4323 Data columns (total 9 columns): # Column Non-Null Count Dtype 0 Unnamed: 0.1 4324 non-null int64 1 Unnamed: 0 4324 non-null int64 20110rganisation 4324 non-null object 3 Location 4324 non-null object 4 Date 4324 non-null object 5 Detail 4324 non-null object 6 Rocket_Status 4324 non-null object 1872Price 1872 9672 non-null object 8 Mission_Status 4324 non-null object dtypes: int64(2), object(7) memory usage: 304.2+ KB</pre>										LA-Y1, Tanegashima Space Center, Japan		Tue Feb 11, 1992 01:50 UTC	H-I (9 SO) JERS 1	StatusRetired	NaN	Success
<pre>LA-Y1, Tanegashima Space Center, Japan</pre>										Thu Feb 19, 1987 01:23 UTC	N-II Momo 1a	StatusRetired	NaN	Success		
<pre>SLC-40, Cape Canaveral AFS, Florida, USA</pre>										Mon Sep 04, 1989 05:54 UTC	Titan 34D DSCS- II-16 & DSCS-III- A2	StatusRetired	NaN	Success		
<pre>Site 31/6, Baikonur</pre>										Sat Nov 25, 1972	Voskhod Cosmos	StatusRetired	NaN	Success		

```
#drop the Price column from the data frame since there are too many missing values
#also drop the two Unnamed columns, which are most likely index columns that were automatically generated
#or preserved during data export or manipulation
#print the first 5 rows of data to confirm that the Price column has been dropped
clean_df_data = df_data.drop(columns=['Price', 'Unnamed: 0.1', 'Unnamed: 0'], axis=1)
clean_df_data.head()
```

Organisation		Location		Sun Jul 1965		Detail		Rocket	Status	Mission	Status
3953	3953	3953	RVSN USSR	Site 1/5, Baikonur Cosmodrome, Russia	07, 2020	05:12 UTC	Falcon 9 Block 5 Starlink V1 L9 & BlackSky	StatusRetired	NaN	Success	Success
0	SpaceX	LC-39A, Kennedy Space Center, Florida, USA			Fri Aug 07, 2020	05:12 UTC	Falcon 9 Block 5 Starlink V1 L9 & BlackSky	StatusActive			Success
3128	CAS	3128	RVSN USSR	Site 41/1, Plesetsk Cosmodrome, Russia	06, 2020	11:50 UTC	Soyuz MS-19 Progress MS-19	StatusRetired	NaN	Success	Success
2	SpaceX	Pad A, Boca Chica, Texas, USA			Tue Aug 04, 2020	23:57 UTC	Starship Prototype 150 Meter Hop	StatusActive			Success
2198	2198	2198	RVSN USSR	Site 1/5, Baikonur Cosmodrome, Russia	08, 2020	11:28 UTC	Proton-M/Breeze-M Cosmos 1570	StatusRetired	NaN	Success	Success

- ✓ Descriptive Statistics

2266	2266	2266	General	SLC-36A, Cape Canaveral AFS,	Thu May 19, 1983	Atlas-SLV3D Centaur-D1AR	StatusRetired	NaN	Success
------	------	------	---------	------------------------------	------------------	--------------------------	---------------	-----	---------

```
#view descriptive statistics for cleaned df
clean_df.describe()
```

2559	2559	2559	RVSN USSR	Site 43/4, Plesetsk Cosmodrome, Russia	17, 1978 15:00 UTC	Soyuz U Cosmos 1044	StatusRetired	NaN	Success
------	------	------	-----------	--	--------------------------	--------------------------	---------------	-----	---------

Organisation		Location		Date		Cosmos-3M	Detail	Rocket_Status	Mission_Status
count	4324	4324	4324	04 Jul 1989	04 Jul 1989		4324	4324	4324
unique	56	137	4319	04 Jul 1989	04 Jul 1989		4278	2	4
top	RVSN USSR	Site 31/6, Baikonur Cosmodrome, Kazakhstan	Tue Aug 28, 1990 09:05 UTC			Cosmos-3MRB (65MRB) BOR-5 Shuttle		StatusRetired	Success
freq	1777	235	2				6	3534	3879

Number of Launches per Company

Create a chart that shows the number of space mission launches by organisation.

```
#save the count of space missions by organization to the variable org_launches
org_launches = clean_df_data.Organisation.value_counts().sort_values(ascending=False)
print(org_launches)
```

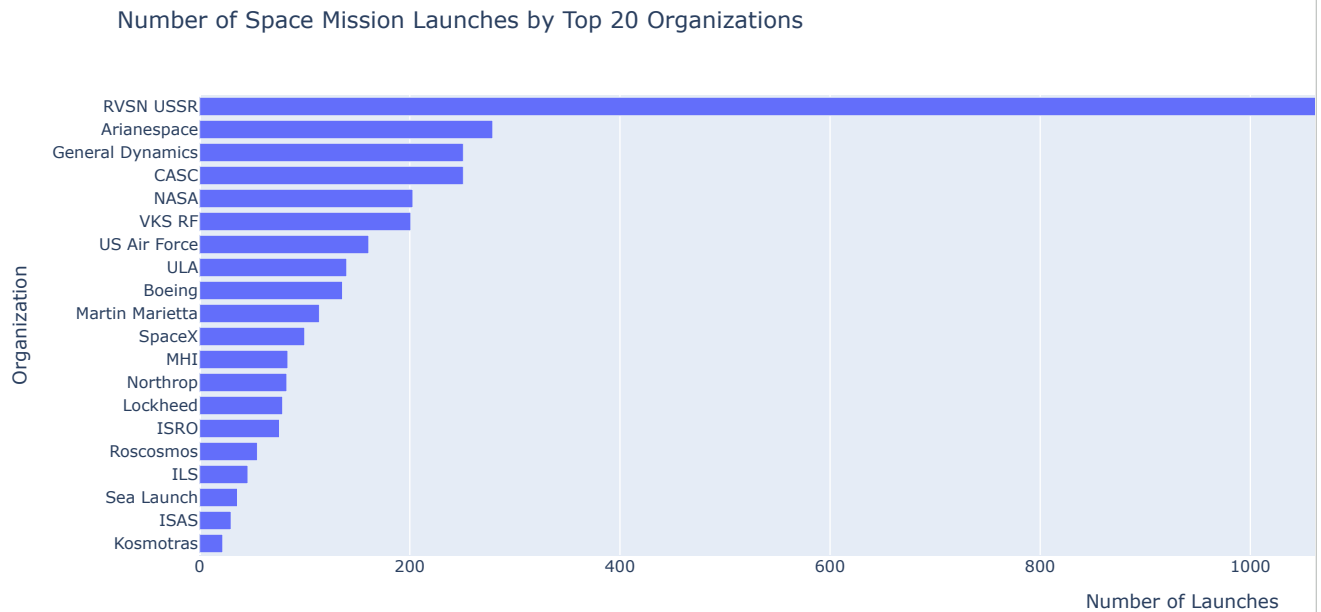
```
Organisation
RVSN USSR      1777
Arianespace    279
General Dynamics 251
CASC            251
NASA           203
VKS RF         201
US Air Force   161
ULA            140
Boeing         136
Martin Marietta 114
SpaceX         100
MHI            84
Northrop       83
Lockheed       79
ISRO           76
Roscosmos      55
ILS            46
Sea Launch     36
ISAS           30
Kosmotras      22
US Navy        17
Rocket Lab     13
ISA            13
Eurokot        13
ESA            13
Blue Origin    12
IAI            11
ExPace         10
ASI            9
CNES           8
AMBA           8
JAXA           7
Land Launch    7
MITT           7
UT             5
CASIC          5
KCST           5
Exos           4
CECLES         4
Arm??e de l'Air 4
SRC            3
KARI           3
AEB            3
RAE            2
Yuzhmash       2
OKB-586        2
Virgin Orbit   1
IRGC           1
OneSpace       1
Landspace      1
i-Space        1
Sandia         1
Starsem        1
Khrunichev     1
Douglas        1
EER            1
Name: count, dtype: int64
```

```
#create a horizontal graph that shows the number of space mission launches by Top 20 organizations.
plt.figure(dpi=200)
```

```

hor = px.bar(x=org_launches.values[:20],
             y=org_launches.index[:20],
             orientation='h',
             title='Number of Space Mission Launches by Top 20 Organizations')
hor.update_layout(xaxis_title='Number of Launches',
                  yaxis_title='Organization',
                  yaxis=dict(autorange='reversed'))
hor.show()

```



<Figure size 1280x960 with 0 Axes>

✓ Number of Active versus Retired Rockets

How many rockets are active compared to those that are decommissioned?

```

rocket_status = clean_df_data['Rocket_Status'].value_counts()
rocket_status

```

	count
Rocket_Status	
StatusRetired	3534
StatusActive	790

dtype: int64

✓ Distribution of Mission Status

How many missions were successful? How many missions failed?

```

mission_status = clean_df_data['Mission_Status'].value_counts()
mission_status

```

	count
Mission_Status	
Success	3879
Failure	339
Partial Failure	102
Prelaunch Failure	4

dtype: int64

Double-click (or enter) to edit

✓ How Expensive are the Launches?

Create a histogram and visualise the distribution. The price column is given in USD millions (careful of missing values).

```
#remove commas and change data in the Price column from object type to numerical type
price = df_data['Price'].copy()
price = df_data['Price'].astype(str).str.replace(',', ' ')
price = pd.to_numeric(df_data['Price'], errors='coerce')

#drop na values
price = price.dropna()

price.info()
```

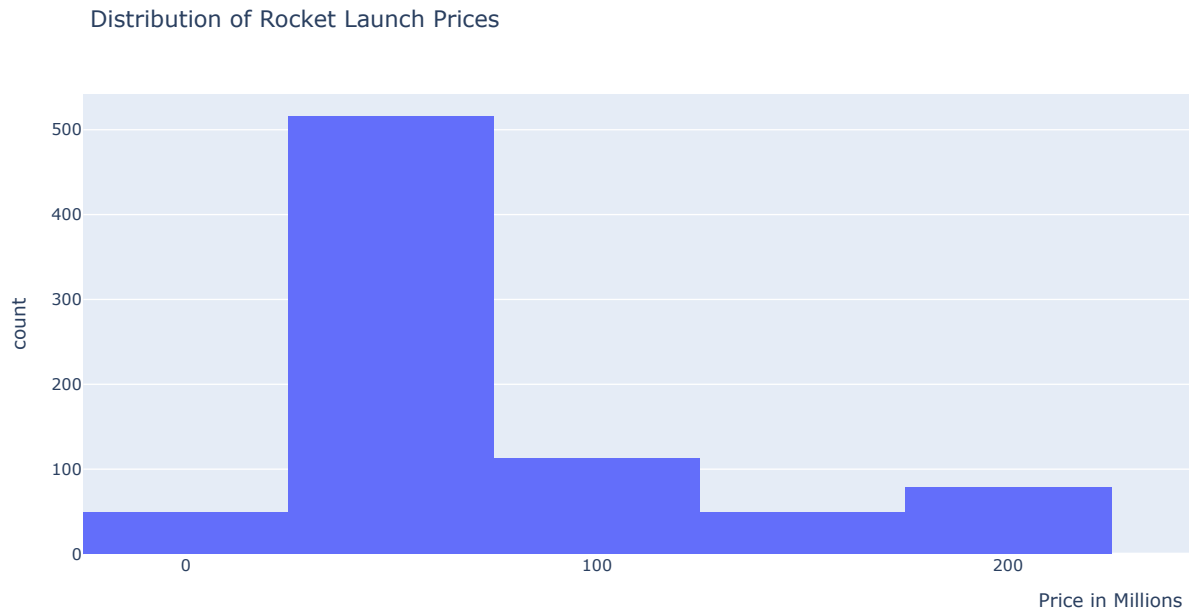
```
<class 'pandas.core.series.Series'>
Index: 949 entries, 0 to 4020
Series name: Price
Non-Null Count  Dtype
-----
949 non-null    float64
dtypes: float64(1)
memory usage: 14.8 KB
```

```
#view the descriptive statistics for price
price.describe()
```

	Price
count	949.00
mean	129.80
std	143.22
min	5.30
25%	40.00
50%	62.00
75%	164.00
max	450.00

dtype: float64

```
#create a histogram to visualize the price distribution for launching rockets
hist = px.histogram(price,
                    nbins=10,
                    title='Distribution of Rocket Launch Prices')
hist.update_layout(xaxis_title='Price in Millions', showlegend=False)
hist.show()
```



✓ Use a Choropleth Map to Show the Number of Launches by Country

- Create a choropleth map using [the plotly documentation](#)
- Experiment with [plotly's available colours](#). I quite like the sequential colour `matter` on this map.
- You'll need to extract a `country` feature as well as change the country names that no longer exist.

Wrangle the Country Names

You'll need to use a 3 letter country code for each country. You might have to change some country names.

- Russia is the Russian Federation
- New Mexico should be USA
- Yellow Sea refers to China
- Shahrud Missile Test Site should be Iran
- Pacific Missile Range Facility should be USA
- Barents Sea should be Russian Federation
- Gran Canaria should be USA

You can use the `iso3166` package to convert the country names to Alpha3 format.

```
from iso3166 import countries_by_name

#custom mapping for renaming countries whose names no longer exist or changed
custom_map = {
    'Russia': 'Russian Federation',
    'New Mexico': 'USA',
    'Yellow Sea': 'China',
    'Shahrud Missile Test Site': 'Iran',
    'Pacific Missile Range Facility': 'USA',
    'Barents Sea': 'Russian Federation',
    'Gran Canaria': 'USA',
}

#Extract the country name from the Location column using inline lambda function and adding Country column to the dataframe
clean_df_data['Country'] = clean_df_data['Location'].apply(lambda x: x.split(",")[-1].strip())

#Apply custom mapping
clean_df_data['Country'] = clean_df_data['Country'].replace(custom_map)
clean_df_data
```

	Organisation	Location	Date	Detail	Rocket_Status	Mission_Status	Country
0	SpaceX	LC-39A, Kennedy Space Center, Florida, USA	Fri Aug 07, 2020 05:12 UTC	Falcon 9 Block 5 Starlink V1 L9 & BlackSky	StatusActive	Success	USA
1	CASC	Site 9401 (SLS-2), Jiuquan Satellite Launch Ce...	Thu Aug 06, 2020 04:01 UTC	Long March 2D Gaofen-9 04 & Q-SAT	StatusActive	Success	China
2	SpaceX	Pad A, Boca Chica, Texas, USA	Tue Aug 04, 2020 23:57 UTC	Starship Prototype 150 Meter Hop	StatusActive	Success	USA
3	Roscosmos	Site 200/39, Baikonur Cosmodrome, Kazakhstan	Thu Jul 30, 2020 21:25 UTC	Proton-M/Briz-M Ekspress-80 & Ekspress-103	StatusActive	Success	Kazakhstan
4	ULA	SLC-41, Cape Canaveral AFS, Florida, USA	Thu Jul 30, 2020 11:50 UTC	Atlas V 541 Perseverance	StatusActive	Success	USA
...	...	...	...	...	...	...	...

#Use iso3166 to convert country names to 3-letter codes and adding Country_Code to the dataframe.

```
def get_alpha3(name):
    try:
        return countries_by_name[name.upper()].alpha3
    except KeyError:
        if name == "USA":
            return "USA"
        else:
            return "UNKNOWN"

clean_df_data["Country_Code"] = clean_df_data["Country"].apply(get_alpha3)
clean_df_data
```

	Organisation	Location	Date	Detail	Rocket_Status	Mission_Status	Country	Country_Code
0	SpaceX	LC-39A, Kennedy Space Center, Florida, USA	Fri Aug 07, 2020 05:12 UTC	Falcon 9 Block 5 Starlink V1 L9 & BlackSky	StatusActive	Success	USA	USA
1	CASC	Site 9401 (SLS-2), Jiuquan Satellite Launch Ce...	Thu Aug 06, 2020 04:01 UTC	Long March 2D Gaofen-9 04 & Q-SAT	StatusActive	Success	China	CHN
2	SpaceX	Pad A, Boca Chica, Texas, USA	Tue Aug 04, 2020 23:57 UTC	Starship Prototype 150 Meter Hop	StatusActive	Success	USA	USA
3	Roscosmos	Site 200/39, Baikonur Cosmodrome, Kazakhstan	Thu Jul 30, 2020 21:25 UTC	Proton-M/Briz-M Ekspress-80 & Ekspress-103	StatusActive	Success	Kazakhstan	KAZ
4	ULA	SLC-41, Cape Canaveral AFS, Florida, USA	Thu Jul 30, 2020 11:50 UTC	Atlas V 541 Perseverance	StatusActive	Success	USA	USA
...	...	...	...	...	...	...	...	...

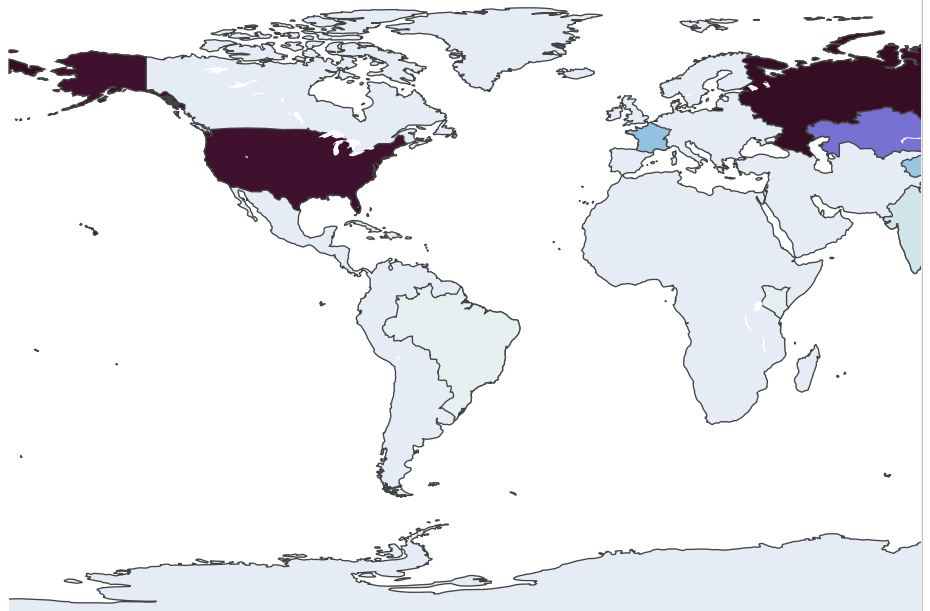
```
#get the count for the number of launches by country
num_launches = clean_df_data['Country_Code'].value_counts().reset_index()
num_launches.columns = ['Country_Code', 'Launch_Count']
num_launches
```

	Country_Code	Launch_Count
0	RUS	1398
1	USA	1351
2	KAZ	701
3	FRA	303
4	CHN	269
5	JPN	126
6	IND	76
7	UNKNOWN	58
8	NZL	13
9	ISR	11
10	KEN	9
11	AUS	6
12	BRA	3

```
#create map that shows the number of rocket launches by country
launch_map = px.choropleth(num_launches,
                           locations='Country_Code',
                           color='Launch_Count',
                           color_continuous_scale='dense',
                           title="Rocket Launches by Country",
                           )
launch_map.update_layout(geo=dict(showframe=False, showcoastlines=True),
                          margin=dict(l=0, r=0, t=50, b=0)
                          )

launch_map.show()
```

Rocket Launches by Country



Use a Choropleth Map to Show the Number of Failures by Country

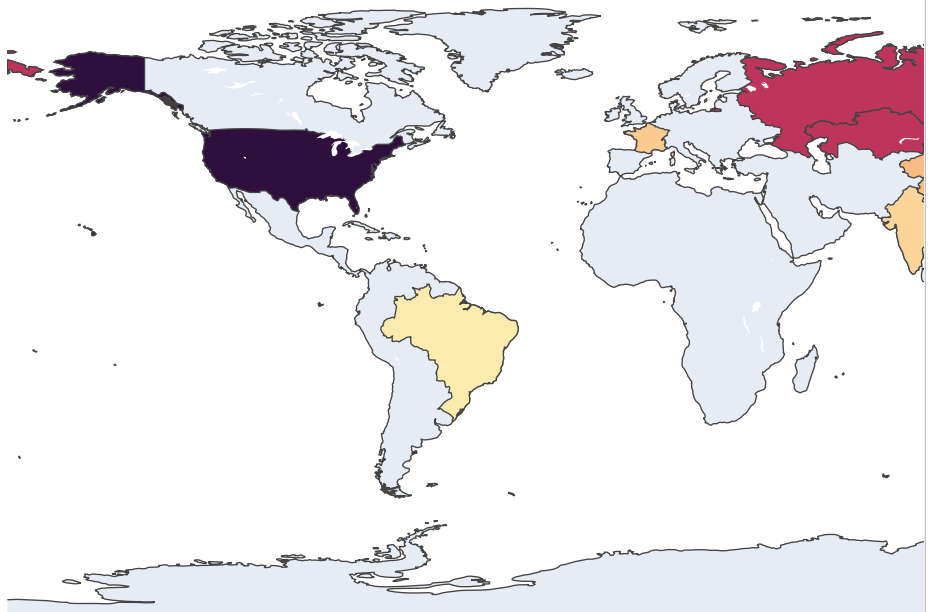
```
#create dataframe that contains only country data for all failed missions
num_failures = clean_df_data['Country_Code'][clean_df_data['Mission_Status'] != 'Success'].value_counts().reset_index()
num_failures.columns=['Country_Code', 'Failed_Missions']
num_failures
```

	Country_Code	Failed_Missions
0	USA	163
1	KAZ	93
2	RUS	93
3	CHN	25
4	FRA	18
5	UNKNOWN	17
6	JPN	13
7	IND	13
8	AUS	3
9	BRA	3
10	NZL	2
11	ISR	2

```
#create map to show number of failed missions by country
failure_map = px.choropleth(num_failures,
                             locations='Country_Code',
                             color='Failed_Missions',
                             color_continuous_scale='matter',
                             title="Failed Missions by Country",
                             )
failure_map.update_layout(geo=dict(showframe=False, showcoastlines=True),
                           margin=dict(l=0, r=0, t=50, b=0)
                           )

failure_map.show()
```

Failed Missions by Country



Create a Plotly Sunburst Chart of the countries, organisations, and mission status.

```
#group data by Country, Organisation, and Mission Status and get the count for the Mission Status
launch_status = clean_df_data.groupby(['Country', 'Organisation', 'Mission_Status']).size().reset_index(name='Count')
print(launch_status[['Country', 'Organisation', 'Mission_Status']])
```

	Country	Organisation	Mission_Status
0	Australia	AMBA	Success
1	Australia	CECLES	Failure
2	Australia	CECLES	Success
3	Australia	RAE	Failure
4	Australia	RAE	Success
..	...	...	...
127	USA	US Air Force	Success
128	USA	US Navy	Failure
129	USA	US Navy	Partial Failure
130	USA	US Navy	Success
131	USA	Virgin Orbit	Failure

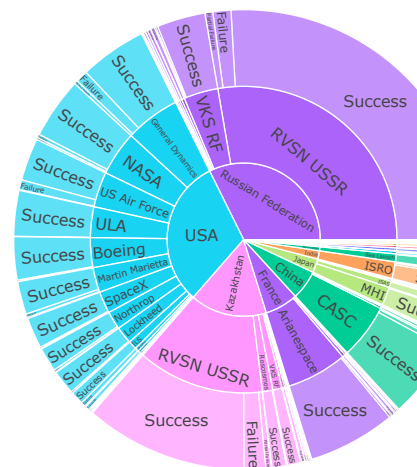
[132 rows x 3 columns]

```
#create a sunburst chart that shows the grouped data by Country, Organisation, and counts of Mission Status
mission_burst = px.sunburst(launch_status,
                             path=['Country', 'Organisation', 'Mission_Status'],
                             values='Count',
                             color='Country',
                             title='Mission Status by Country and Organization?')
```

```
mission_burst.update_layout(xaxis_title='Mission Status',
                             yaxis_title='Country',
                             coloraxis_showscale=False)
```

```
mission_burst.show()
```

Mission Status by Country and Organization?



Analyse the Total Amount of Money Spent by Organisation on Space Missions

```
#create a new dataframe containing organization and price
price_df = df_data[['Organisation', 'Price']].copy()

#clean price column by removing comma and converting to numeric
price_df['Price'] = price_df['Price'].astype(str).str.replace(",", "")
price_df['Price'] = pd.to_numeric(price_df['Price'], errors='coerce')

# Drop rows where Price is NaN
```

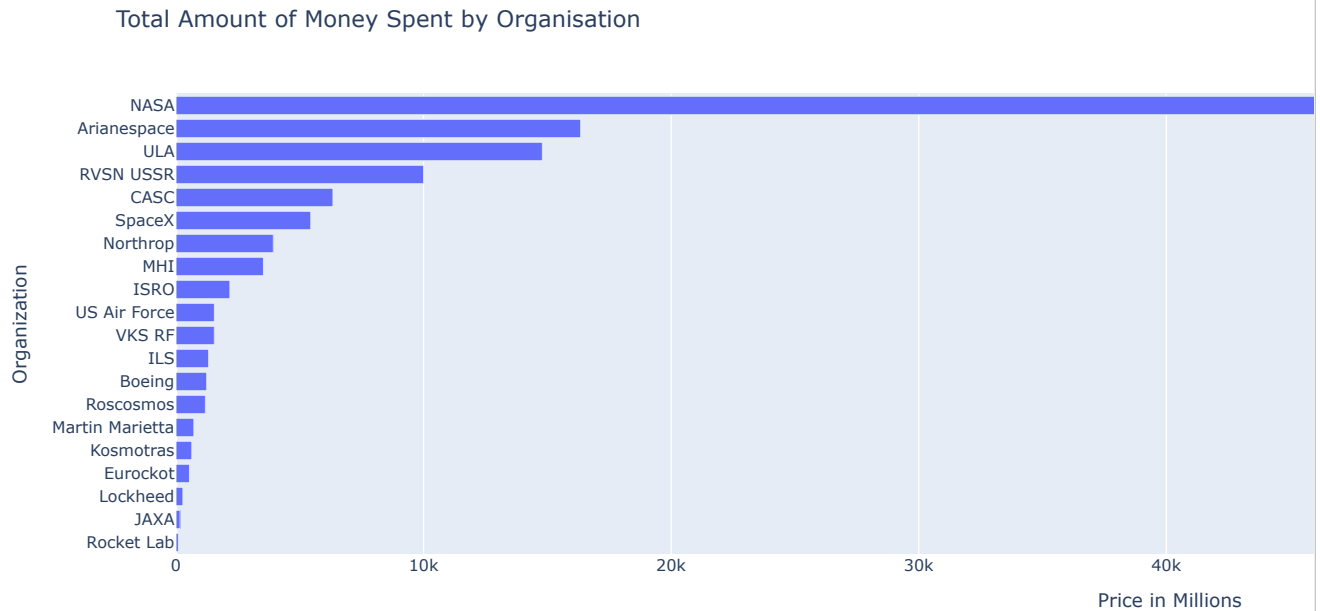
```
price_df = price_df.dropna(subset=['Price'])
```

```
#get the total money spent by organization
```

```
money_spent = price_df.groupby(['Organisation'], as_index=False).agg({'Price': 'sum'}).sort_values(by='Price', ascending=False)
money_spent
```

	Organisation	Price
14	NASA	76,280.00
0	Arianespace	16,345.00
21	ULA	14,798.00
16	RVSN USSR	10,000.00
2	CASC	6,340.26
20	SpaceX	5,444.00
15	Northrop	3,930.00
12	MHI	3,532.50
8	ISRO	2,177.00
22	US Air Force	1,550.92
23	VKS RF	1,548.90
7	ILS	1,320.00
1	Boeing	1,241.00
18	Roscosmos	1,187.50
13	Martin Marietta	721.40
10	Kosmotras	638.00
5	Eurokot	543.40
11	Lockheed	280.00
9	JAXA	168.00
17	Rocket Lab	97.50
4	ESA	37.00
6	ExPace	28.30
3	EER	20.00
19	Sandia	15.00
24	Virgin Orbit	12.00

```
total_spent = px.bar(x=money_spent.Price[:20],
                    y=money_spent.Organisation[:20],
                    title='Total Amount of Money Spent by Organisation')
total_spent.update_layout(xaxis_title='Price in Millions',
                          yaxis_title='Organization',
                          yaxis=dict(autorange='reversed'))
total_spent.show()
```



✓ Analyse the Amount of Money Spent by Organisation per Launch

```
#create a new dataframe containing organization and price
price_launch_df = df_data[['Organisation', 'Price', 'Mission_Status']].copy()

#clean price column by removing comma and converting to numeric
price_launch_df['Price'] = price_launch_df['Price'].astype(str).str.replace(",", "")
price_launch_df['Price'] = pd.to_numeric(price_launch_df['Price'], errors='coerce')

# Drop rows where Price is NaN
price_launch_df = price_launch_df.dropna(subset=['Price'])

#get the total money spent and the total number of missions by organization
money_launch_spent = price_launch_df.groupby(['Organisation'], as_index=False).agg({'Price': 'sum', 'Mission_Status': 'count'})

money_launch_spent['Price_Per_Launch'] = money_launch_spent.Price/money_launch_spent.Mission_Status
money_launch_spent_sorted = money_launch_spent.sort_values(by='Price_Per_Launch', ascending=False)
money_launch_spent_sorted
```

	Organisation	Price	Mission_Status	Price_Per_Launch
16	RVSN USSR	10,000.00	2	5,000.00
14	NASA	76,280.00	149	511.95
1	Boeing	1,241.00	7	177.29
0	Arianespace	16,345.00	96	170.26
21	ULA	14,798.00	98	151.00
7	ILS	1,320.00	13	101.54
12	MHI	3,532.50	37	95.47
13	Martin Marietta	721.40	9	80.16
22	US Air Force	1,550.92	26	59.65
9	JAXA	168.00	3	56.00
20	SpaceX	5,444.00	99	54.99
18	Roscosmos	1,187.50	23	51.63
15	Northrop	3,930.00	83	47.35
23	VKS RF	1,548.90	33	46.94
5	Eurokot	543.40	13	41.80
2	CASC	6,340.26	158	40.13
4	ESA	37.00	1	37.00
11	Lockheed	280.00	8	35.00
8	ISRO	2,177.00	67	32.49
10	Kosmotras	638.00	22	29.00
6	ExPace	28.30	1	28.30
3	EER	20.00	1	20.00
19	Sandia	15.00	1	15.00
24	Virgin Orbit	12.00	1	12.00
17	Rocket Lab	97.50	13	7.50

```

price_per_launch = px.bar(x=money_launch_spent_sorted.Price_Per_Launch,
                           y=money_launch_spent_sorted.Organisation,
                           title=' Amount of Money Spent Per Launch',
                           color_discrete_sequence=['#2ecc71'])
price_per_launch.update_layout(xaxis_title='Price Per Launch',
                               yaxis_title='Organization',
                               yaxis=dict(autorange='reversed'))
price_per_launch.show()

```

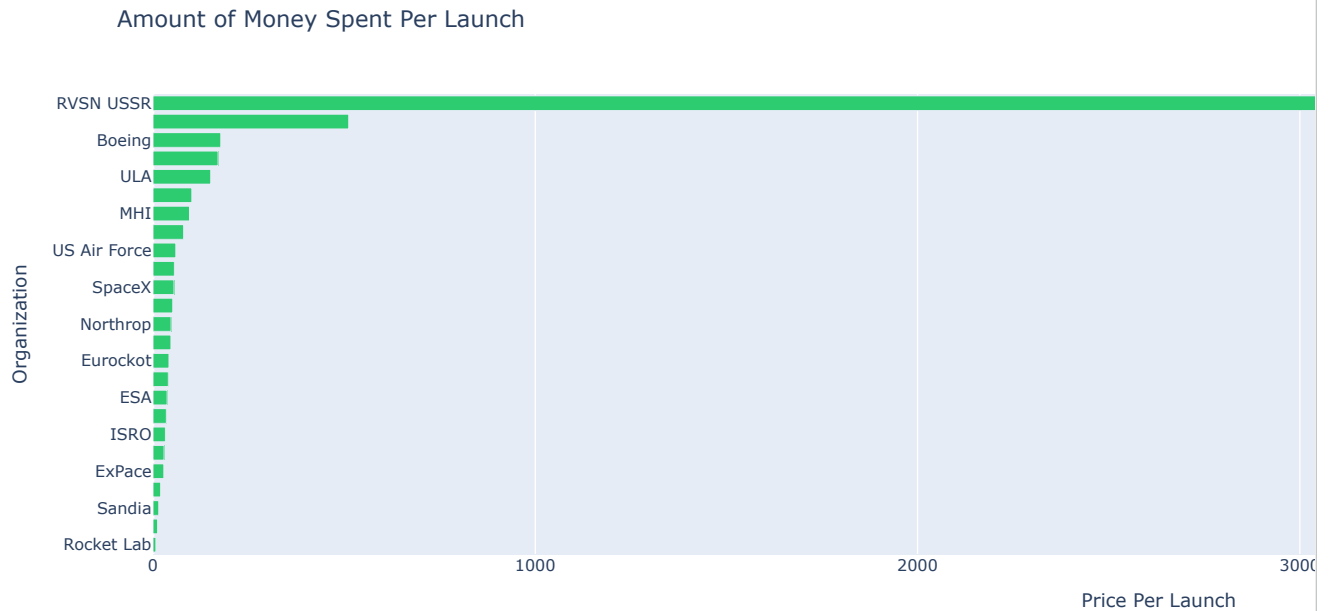


Chart the Number of Launches per Year

```
#create a new data frame date_df
date_df = clean_df_data.copy()
```

```
#remove UTC to standardize Date column
date_df['Date'] = date_df['Date'].astype(str).str.replace('UTC', '')
```

```
#Convert Date column from object to datetime format and view column info for confirmation
date_df['Date'] = pd.to_datetime(date_df['Date'], errors='coerce')
date_df['Date'].info()
```

```
<class 'pandas.core.series.Series'>
RangeIndex: 4324 entries, 0 to 4323
Series name: Date
Non-Null Count  Dtype
-----  -----
4198 non-null   datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 33.9 KB
```

```
#check fo NAN values post datetime conversion
date_df['Date'].isna().values.sum()
```

```
np.int64(126)
```

```
#check fo NAN values post datetime conversion
date_df['Date'].isna().value_counts()
```

```
count
Date
False  4198
True   126

dtype: int64
```

```
#Tried different formatting of Date column to try to mitigate data being converted to NAN post datetime conversion.
#Since could not find any solution even after research, will drop rows with NAN values
date_df = date_df.dropna(subset=['Date'])
```

```
#abstract only the year from the Date column and create a new column "year"
date_df['Year'] = date_df['Date'].dt.year
```

```
#cheack for NAN values in the year column
date_df['Year'].isna().values.sum()
```

```
np.int64(0)
```

```
#print 30 random rows to check that the Date and Year column formatting was successful
date_df.sample(30)
```

	Organisation	Location	Date	Detail	Rocket_Status	Mission_Status	Country	Country_Code	Year
1946	RVSN USSR	Site 90/19, Baikonur Cosmodrome, Kazakhstan	1988-05-28 02:49:00	Tsyklon-2 Cosmos 1949	StatusRetired	Success	Kazakhstan	KAZ	1988
2634	RVSN USSR	Site 32/2, Plesetsk Cosmodrome, Russia	1977-12-27 08:00:00	Tsyklon-3 Cosmos 972	StatusRetired	Success	Russian Federation	RUS	1977
728	Roscosmos	Site 1/5, Baikonur Cosmodrome, Kazakhstan	2010-04-28 17:15:00	Soyuz U Progress M-05M	StatusRetired	Success	Kazakhstan	KAZ	2010
1380	Boeing	SLC-17A, Cape Canaveral AFS, Florida, USA	1997-07-23 03:43:00	Delta II 7925 USA-132 (GPS IIR-2)	StatusRetired	Success	USA	USA	1997
4227	US Air Force	SLC-1E (75-3-5), Vandenberg AFB, California, USA	1961-02-18 22:58:00	Thor DM-21 Agena-B Discoverer 21	StatusRetired	Success	USA	USA	1961
1144	NASA	LC-39A, Kennedy Space Center, Florida, USA	2001-04-19 18:40:00	Space Shuttle Endeavour STS-100	StatusRetired	Success	USA	USA	2001
179	ISRO	Second Launch Pad, Satish Dhawan Space Centre,...	2018-12-19 10:40:00	GSLV Mk II GSAT-7A	StatusActive	Success	India	IND	2018
1478	Arianespace	ELA-2, Guiana Space Centre, French Guiana, France	1995-11-17 01:20:00	Ariane 44P Infrared Space Observatory	StatusRetired	Success	France	FRA	1995
1318	Arianespace	ELA-2, Guiana Space Centre, French Guiana, France	1998-04-28 22:53:00	Ariane 44P Nilesat-101, BSAT-1B	StatusRetired	Success	France	FRA	1998
737	NASA	LC-39A, Kennedy Space Center, Florida, USA	2010-02-08 09:14:00	Space Shuttle Endeavour STS-130	StatusRetired	Success	USA	USA	2010
913	ISRO	Second Launch Pad, Satish Dhawan Space Centre,...	2006-07-10 12:08:00	GSLV Mk I INSAT-4C	StatusRetired	Failure	India	IND	2006
884	CASC	LC-2, Xichang Satellite Launch Center, China	2007-02-02 16:28:00	Long March 3A Beidou-1D	StatusActive	Success	China	CHN	2007
3464	Roscosmos	Site 31/6, Baikonur Cosmodrome, Kazakhstan	1970-06-01 19:00:00	Soyuz Soyuz 9	StatusRetired	Success	Kazakhstan	KAZ	1970

```
#create a dataframe that contains the count of lauches per year
launch_per_year = date_df.groupby('Year').size().reset_index(name='Launches')
launch_per_year
```

	Year	Launches
0	1957	3
1	1958	22
2	1959	20
3	1960	38
4	1961	52
...	...	...
59	2016	88
60	2017	90
61	2018	117
62	2019	107
63	2020	63

64 rows × 2 columns

```
#graph the number of launches per year
plt.figure(figsize=(8,6), dpi=200)

plt.xlabel('Date', fontsize=12)
plt.ylabel('Number of Launches', fontsize=12)
plt.title('Number of Launches Per Year', fontsize=14)

plt.plot(launch_per_year.Year, launch_per_year.Launches)
plt.show()
```

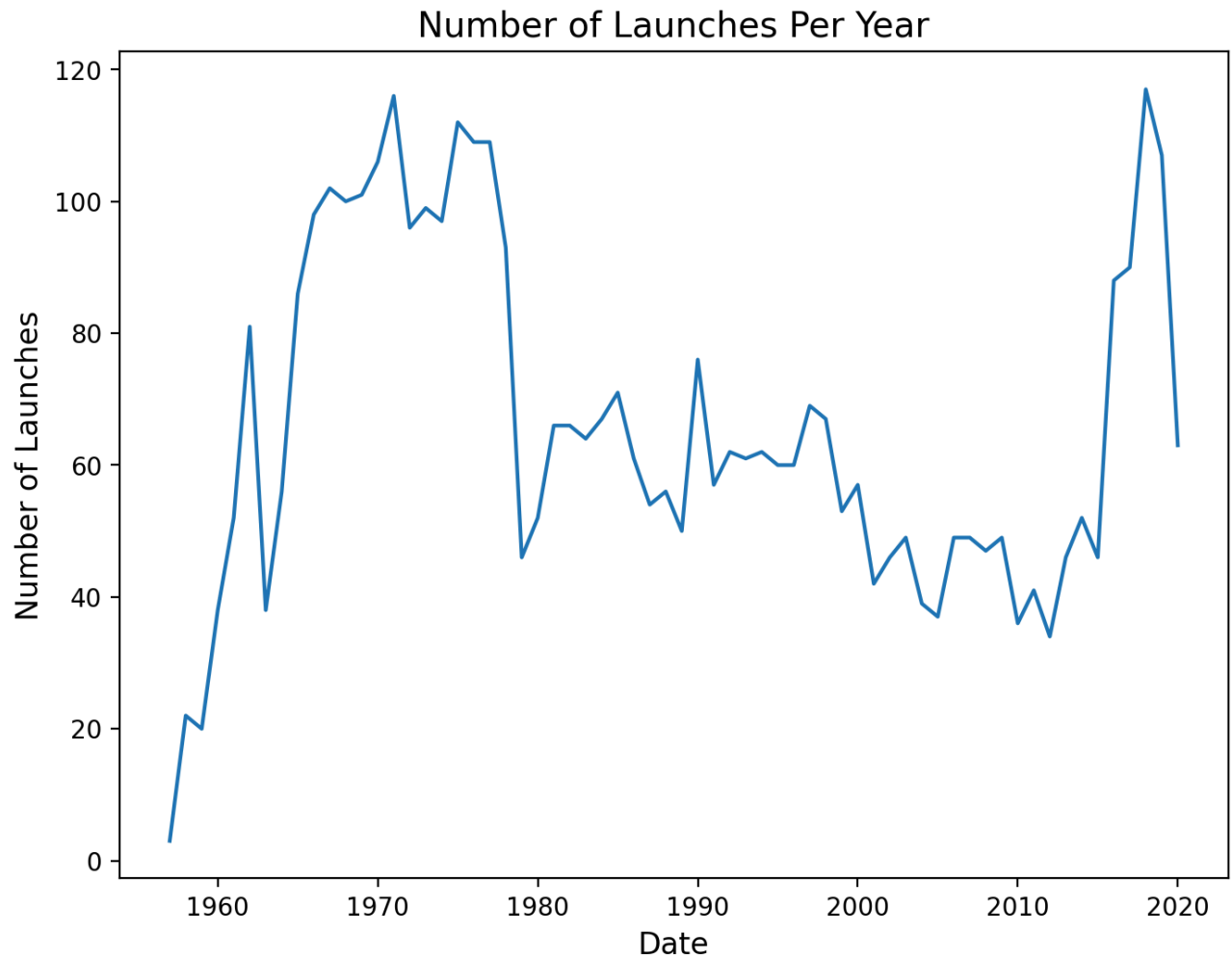


Chart the Number of Launches Month-on-Month until the Present

Which month has seen the highest number of launches in all time? Superimpose a rolling average on the month on month time series chart.

```
#create a new column for yearmonth data
date_df['YearMonth'] = date_df['Date'].dt.to_period('M')

#view data to confirm column was successfully added
date_df.head()
```

	Organisation	Location	Date	Detail	Rocket_Status	Mission_Status	Country	Country_Code	Year	YearMonth
0	SpaceX	LC-39A, Kennedy Space Center, Florida, USA	2020-08-07 05:12:00	Falcon 9 Block 5 Starlink V1 L9 & BlackSky	StatusActive	Success	USA	USA	2020	2020-08
1	CASC	Site 9401 (SLS-2), Jiuquan Satellite	2020-08-06 04:01:00	Long March 2D Gaofen-9 04 & Q-SAT	StatusActive	Success	China	CHN	2020	2020-08

```
date_df['YearMonth'].info()
```

```
<class 'pandas.core.series.Series'>
Index: 4198 entries, 0 to 4323
Series name: YearMonth
Non-Null Count  Dtype
-----
4198 non-null   period[M]
dtypes: period[M](1)
memory usage: 65.6 KB
```

```
#Convert YearMonth to datetime object(e.g. timestamp representing the start of the month)
date_df['YearMonth'] = date_df['YearMonth'].dt.to_timestamp()
```

```
date_df['YearMonth'].info()
```

```
<class 'pandas.core.series.Series'>
Index: 4198 entries, 0 to 4323
Series name: YearMonth
Non-Null Count  Dtype
-----
4198 non-null   datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 65.6 KB
```

```
#cheack for NAN values in the monthyear column
date_df['YearMonth'].isna().values.sum()
```

```
np.int64(0)
```

```
#create a dataframe that contains the number of launches month to month
launch_month_to_month = date_df.groupby('YearMonth').size().reset_index(name='Launches')
launch_month_to_month = launch_month_to_month.sort_values('YearMonth')
launch_month_to_month
```

	YearMonth	Launches
0	1957-10-01	1
1	1957-11-01	1
2	1957-12-01	1
3	1958-02-01	2
4	1958-03-01	3
...	...	...
742	2020-04-01	5
743	2020-05-01	9
744	2020-06-01	7
745	2020-07-01	14
746	2020-08-01	3

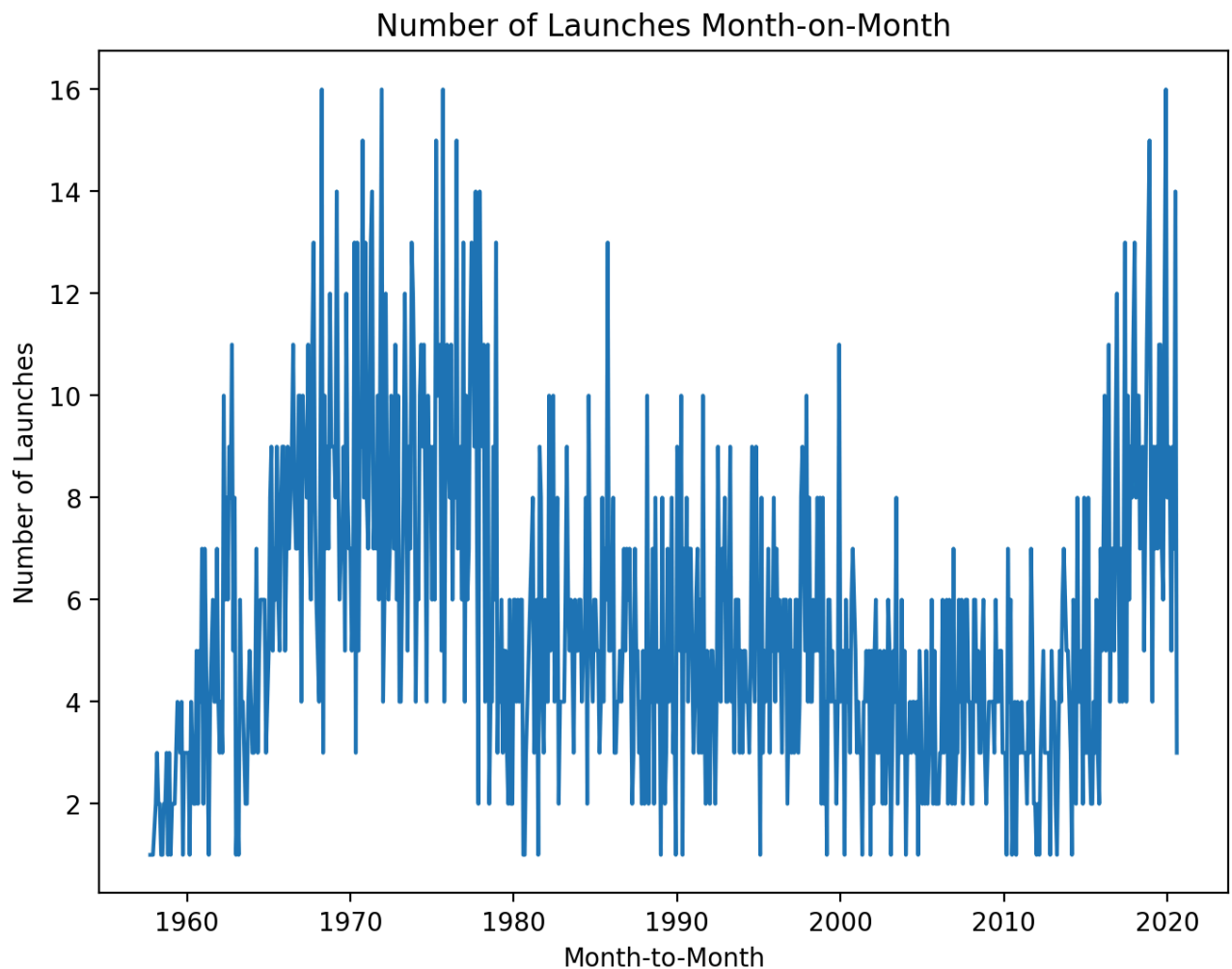
747 rows × 2 columns

```
#Graph the Number of Launches Month-on-Month until the Present
plt.figure(figsize=(8,6), dpi=200)
```

```
plt.xlabel('Month-to-Month')
plt.ylabel('Number of Launches')
plt.title('Number of Launches Month-on-Month')
```

```
plt.plot(launch_month_to_month.YearMonth, launch_month_to_month.Launches)
```

```
plt.show()
```



```
#calculate the rolling mean for the number of launches month-to-month
rolling_month = launch_month_to_month['Launches'].rolling(window=6).mean()
```

```
#Graph the Number of Launches Month-on-Month until the Present and superimpose a rolling average on the month on month time ser

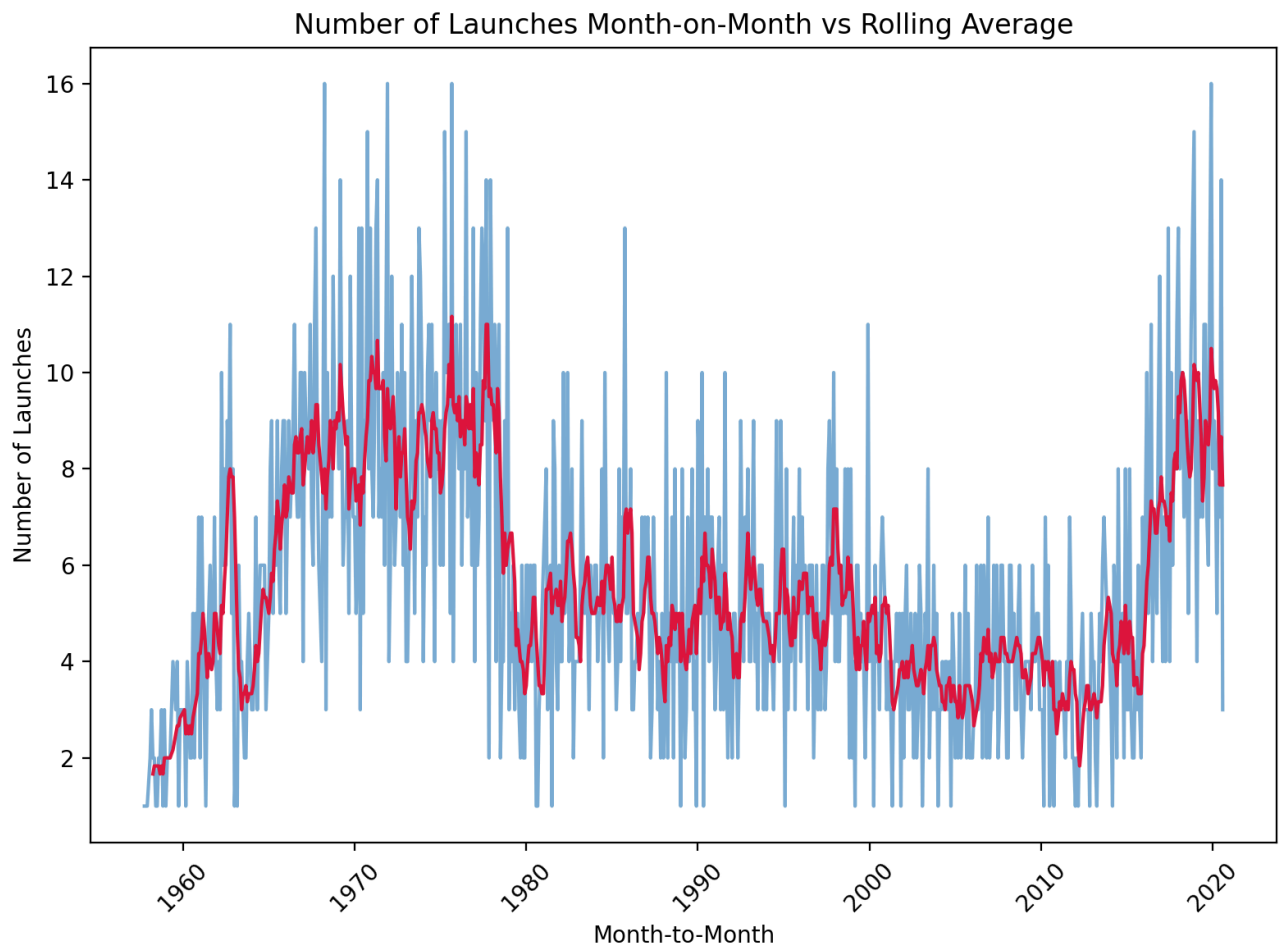
plt.figure(figsize=(8,6), dpi=200)

plt.xlabel('Month-to-Month')
plt.ylabel('Number of Launches')
plt.title('Number of Launches Month-on-Month vs Rolling Average')

plt.plot(launch_month_to_month.YearMonth, launch_month_to_month.Launches, alpha=0.6)
plt.plot(launch_month_to_month.YearMonth, rolling_month, color='crimson')

plt.xticks(rotation=45) # rotate labels for readability
plt.tight_layout()

plt.show()
```



Launches per Month: Which months are most popular and least popular for launches?

Some months have better weather than others. Which time of year seems to be best for space missions?

```
# Extract full month name directly from the Date column and create a new column "Month_Name"
date_df['Month_Name'] = date_df['Date'].dt.strftime('%B')
```

```
#check Month_Name column for NAN values
date_df['Month_Name'].isna().values.sum()
```

```
np.int64(0)
```

```
#print 30 random rows to check that the Month_Name column formatting was successful
date_df.sample(30)
```

	Organisation	Location	Date	Detail	Rocket_Status	Mission_Status	Country	Country_Code	Year	YearMonth	Month
2295	General Dynamics	SLC-3W, Vandenberg AFB, California, USA	1982-12-21 02:38:00	Atlas-E/F Star-37S-ISS DMSP 5D-2/F6	StatusRetired	Success	USA	USA	1982	1982-12-01	
847	NASA	LC-39A, Kennedy Space Center, Florida, USA	2007-10-23 03:38:00	Space Shuttle Discovery STS-120	StatusRetired	Success	USA	USA	2007	2007-10-01	
1248	Boeing	SLC-17B, Cape Canaveral AFS, Florida, USA	1999-06-10 13:48:00	Delta II 7420-10C Globalstar 3	StatusRetired	Success	USA	USA	1999	1999-06-01	
4320	AMBA	LC-26A, Cape Canaveral AFS, Florida, USA	1958-02-01 03:48:00	Juno I Explorer 1	StatusRetired	Success	USA	USA	1958	1958-02-01	
3353	RVSN USSR	Site 31/6, Baikonur Cosmodrome, Kazakhstan	1971-05-06 06:20:00	Voskhod Cosmos 410	StatusRetired	Success	Kazakhstan	KAZ	1971	1971-05-01	
2774	RVSN USSR	Site 31/6, Baikonur Cosmodrome, Kazakhstan	1976-10-10 09:35:00	Soyuz U Cosmos 859	StatusRetired	Success	Kazakhstan	KAZ	1976	1976-10-01	
3164	RVSN USSR	Site 41/1, Plesetsk Cosmodrome, Russia	1973-03-01 12:40:00	Voskhod Cosmos 550	StatusRetired	Success	Russian Federation	RUS	1973	1973-03-01	
3223	Martin Marietta	SLC-4E, Vandenberg AFB, California, USA	1972-07-07 17:46:00	Titan IIID KH-9 & SSF-B-23	StatusRetired	Success	USA	USA	1972	1972-07-01	
3051	Martin Marietta	SLC-4E, Vandenberg AFB, California, USA	1974-04-10 20:20:00	Titan IIID KH-9, SSF-B-25, IRCB	StatusRetired	Success	USA	USA	1974	1974-04-01	
2934	Martin Marietta	SLC-40, Cape Canaveral AFS, Florida, USA	1975-05-20 14:03:00	Titan III(23)C DSCS-II 5&6	StatusRetired	Failure	USA	USA	1975	1975-05-01	

```
#Get the count of launches for each month
month_counts = date_df['Month_Name'].value_counts()
month_counts
```

Month_Name	count
December	430
June	386
October	375
April	366
August	358
September	354
March	349
July	347
November	325
February	323
May	320
January	265

dtype: int64

```
#Determine the most popular and the least popular month for launches
most_popular = month_counts.idxmax()
least_popular = month_counts.idxmin()
print(f"The most popular month for launches is {most_popular} ")
print(f"The least popular month for launches is {least_popular} ")
```

The most popular month for launches is December
The least popular month for launches is January

```
#Determine the top 3 most popular and the top 3 least popular months for launches
Top_3_most_popular = month_counts.head(3).index.to_list()
print(f"The Top_3_most_popular months for launches are {Top_3_most_popular}")

Top_3_least_popular = month_counts.tail(3).index.to_list()
print(f"The Top_3_least_popular months for launches are {Top_3_least_popular}")
```

The Top_3_most_popular months for launches are ['December', 'June', 'October']
The Top_3_least_popular months for launches are ['February', 'May', 'January']

✓ How has the Launch Price varied Over Time?

Create a line chart that shows the average price of rocket launches over time.

```
#create a new dataframe containing date and price
price_change_df = df_data[['Date', 'Price']].copy()

#clean price column by removing comma and converting to numeric
price_change_df['Price'] = price_change_df['Price'].astype(str).str.replace(",", "")
price_change_df['Price'] = pd.to_numeric(price_change_df['Price'], errors='coerce')

# Drop rows where Price is NaN
price_change_df = price_change_df.dropna(subset=['Price'])
```

```
#remove UTC to standardize Date column
price_change_df['Date'] = price_change_df['Date'].astype(str).str.replace('UTC', '')

#covert Date column to datetime
price_change_df['Date'] = pd.to_datetime(price_change_df['Date'], errors='coerce')

#view column issue to confirm conversion
price_change_df['Date'].info()
```

```
<class 'pandas.core.series.Series'>
Index: 964 entries, 0 to 4020
Series name: Date
Non-Null Count  Dtype
-----  -----
```

```
958 non-null    datetime64[ns]
dtypes: datetime64[ns](1)
memory usage: 15.1 KB
```

```
#check for NAN values in Date column post conversion
price_change_df['Date'].isna().values.sum()
```

```
np.int64(6)
```

```
#check for NAN values in Date column post conversion
price_change_df['Date'].isna().value_counts()
```

	count
Date	
False	958
True	6

```
dtype: int64
```

```
#Tried different formatting of Date column to try to mitigate data being converted to NAN post datetime conversion.
#Since could not find any solution even after research, will drop rows with NAN values
price_change_df = price_change_df.dropna(subset=['Date'])
```

```
#abstract only the year from the Date column and create a new column "year"
price_change_df['Year'] = price_change_df['Date'].dt.year
```

```
#check for NAN values in the year column
price_change_df['Year'].isna().values.sum()
```

```
np.int64(0)
```

```
#print 30 random rows to confirm the year column was successfully added and formatted
price_change_df.sample(30)
```

	Date	Price	Year
47	2020-02-19 21:07:00	29.75	2020
1290	1998-09-23 05:06:00	40.00	1998
1408	1997-01-12 09:27:00	450.00	1997
682	2011-07-08 15:29:00	450.00	2011
791	2008-11-15 00:55:00	450.00	2008
2233	1983-11-28 16:00:00	450.00	1983
371	2017-02-19 14:39:00	62.00	2017
466	2016-01-27 23:20:00	200.00	2016
713	2010-09-08 03:30:00	41.80	2010
870	2007-05-25 07:12:00	29.75	2007
1250	1999-05-27 10:49:00	450.00	1999
276	2018-01-31 21:25:00	62.00	2018
114	2019-08-06 19:30:00	200.00	2019
876	2007-04-17 06:46:00	29.00	2007
3815	1966-12-14 18:14:00	59.00	1966
1705	1992-03-24 13:13:00	450.00	1992
878	2007-04-11 03:27:00	30.80	2007
693	2011-03-05 22:46:00	120.00	2011
1421	1996-09-16 08:54:00	450.00	1996
1092	2002-06-05 21:22:00	450.00	2002
770	2009-05-14 13:12:00	200.00	2009
115	2019-08-05 21:56:00	65.00	2019
2348	1982-03-22 16:00:00	450.00	1982
688	2011-05-07 18:10:00	109.00	2011
823	2008-04-25 15:35:00	20.14	2008
1080	2002-09-12 10:23:00	25.00	2002
2180	1984-08-30 12:41:00	450.00	1984
863	2007-06-28 15:02:00	29.00	2007
768	2009-06-18 17:32:00	123.00	2009
715	2010-08-04 20:45:00	200.00	2010

```
#get the total price of launches per year
price_change_time = price_change_df.groupby(['Year'], as_index=False).agg({'Price':'mean'})
price_change_time
```

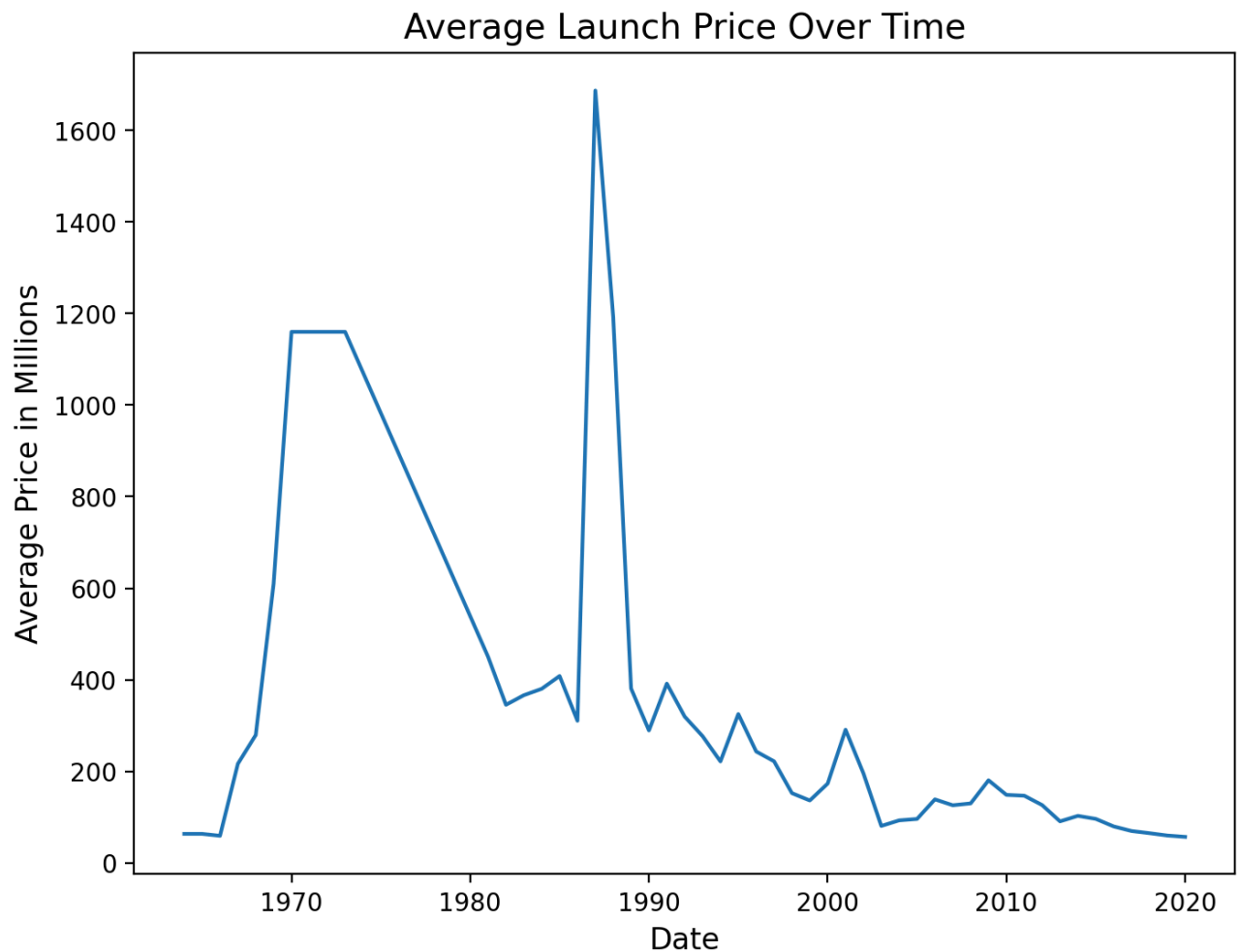
	Year	Price
0	1964	63.23
1	1965	63.23
2	1966	59.00
3	1967	216.29
4	1968	279.20
5	1969	609.50
6	1970	1,160.00
7	1971	1,160.00
8	1972	1,160.00
9	1973	1,160.00
10	1981	450.00
11	1982	345.20
12	1983	366.16
13	1984	380.13
14	1985	408.08
15	1986	310.27
16	1987	1,687.20
17	1988	1,193.16
18	1989	380.83
19	1990	289.15
20	1991	391.43
21	1992	319.35
22	1993	276.73
23	1994	221.51
24	1995	325.00
25	1996	243.20
26	1997	221.74
27	1998	152.12
28	1999	136.17
29	2000	173.12
30	2001	290.70

```
#graph to show how launch price varied over time
plt.figure(figsize=(8,6), dpi=200)
```

```
plt.xlabel('Date', fontsize=12)
plt.ylabel('Average Price in Millions', fontsize=12)
plt.title('Average Launch Price Over Time', fontsize=14)

plt.plot(price_change_time.Year, price_change_time.Price)
plt.show()
```

37	2008	129.75
38	2009	180.29
39	2010	148.40
40	2011	146.60
41	2012	126.10
42	2013	90.55
43	2014	102.55
44	2015	95.95
45	2016	79.46



✓ Chart the Number of Launches over Time by the Top 10 Organisations.

How has the dominance of launches changed over time between the different players?

```
#create a dataframe that contains the count of lauches over time by the top 10 organizations

#group data by organization and year
launch_count = date_df.groupby(['Organisation', 'Year']).size().reset_index(name='Launches')

#find the top ten organizations
top_ten_orgs = date_df['Organisation'].value_counts().nlargest(10).index

#filter data to include only the top ten organizations
launch_over_time = launch_count[launch_count['Organisation'].isin(top_ten_orgs)]

#view dataframe
launch_over_time
```

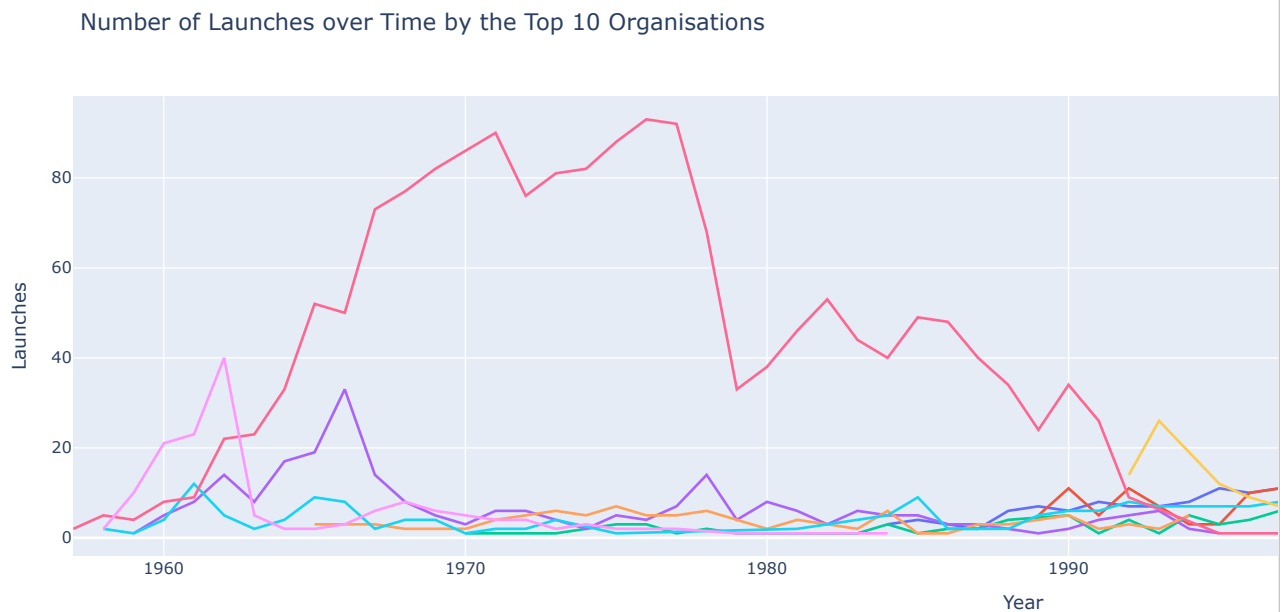
	Organisation	Year	Launches
9	Arianespace	1984	3
10	Arianespace	1985	4
11	Arianespace	1986	3
12	Arianespace	1987	2
13	Arianespace	1988	6
...	...	...	...
623	VKS RF	2016	4
624	VKS RF	2017	6
625	VKS RF	2018	5
626	VKS RF	2019	8
627	VKS RF	2020	3

322 rows × 3 columns

```
#graph to show how launch price varied over time
plt.figure(figsize=(8,6), dpi=200)

graph = px.line(launch_over_time,
                x='Year',
                y='Launches',
                color='Organisation',
                title='Number of Launches over Time by the Top 10 Organisations')

graph.show()
```



<Figure size 1600x1200 with 0 Axes>

✓ Cold War Space Race: USA vs USSR

The cold war lasted from the start of the dataset up until 1991.

```
#filter data to include data up to 1991
cold_war = date_df[date_df['Year'] <= 1991]

#get launch count for the USA
usa_count = cold_war[cold_war['Country'] == 'USA'].shape[0]
```

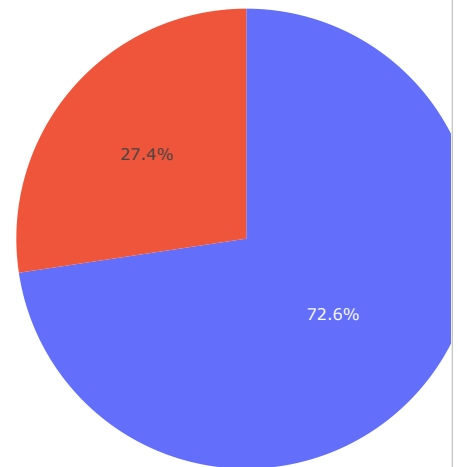
```
#get launch count for the Russian Federation and Kazakhstan
ussr_count = cold_war[cold_war['Country'].isin(['Russian Federation', 'Kazakhstan'])].shape[0]
```

✓ Create a Plotly Pie Chart comparing the total number of launches of the USSR and the USA

Hint: Remember to include former Soviet Republics like Kazakhstan when analysing the total number of launches.

```
#create a pie chart comparing the total number of launches of the USSR and the USA
pie_chart = px.pie(labels=['USA', 'USSR'], values=[usa_count, ussr_count], title='Cold War Space Race: USA vs USSR', names=['US
pie_chart.show()
```

Cold War Space Race: USA vs USSR



✓ Create a Chart that Shows the Total Number of Launches Year-On-Year by the Two Superpowers

```
# Create a new column that merges RUS and KAZ into a single group
cold_war['Country_Group'] = cold_war['Country_Code'].replace({'RUS': 'USSR', 'KAZ': 'USSR'})

# Group by Year and the new Country_Group column. "unstack" reshapes the data for plotting by pivoting the inner index level (C
#"fill_value=0" replaces all missing values with 0, since not every year has launches from every country.
launch_counts = cold_war.groupby(['Year', 'Country_Group']).size().unstack(fill_value=0)

#poy graph using matplotlib
ax = launch_counts[['USSR', 'USA']].plot(kind='line', figsize=(10, 6))

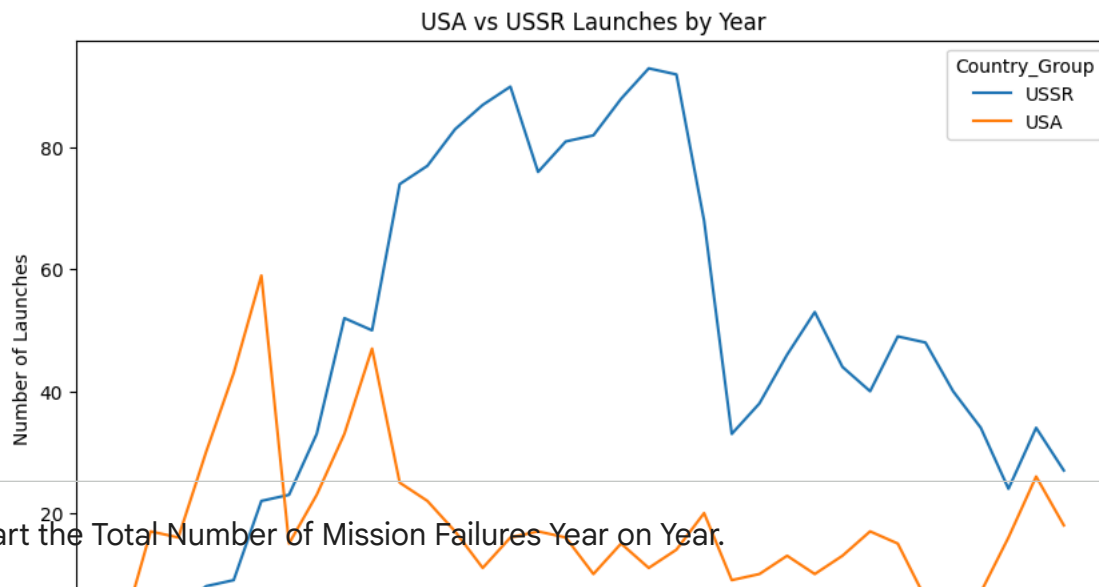
ax.set_ylabel("Number of Launches")
ax.set_title("USA vs USSR Launches by Year")

plt.show()
```

/tmp/ipython-input-3378681071.py:2: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver



✓ Chart the Total Number of Mission Failures Year on Year.

```
#filter data to include only failed missions
mission_failures = cold_war[cold_war['Mission_Status']=="Failure"]

#group failures by year and country
failures_per_year = mission_failures.groupby(['Year', 'Country_Group']).size().unstack(fill_value=0)

#plot graph
ax = failures_per_year[['USSR', 'USA']].plot(kind='line', figsize=(10, 6))

ax.set_ylabel('Mission Failures')
ax.set_title('USA vs USSR Mission Failures Per Year')

plt.show()
```

